LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

M. Rifky Almadani NIM. 2510817310018

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : M. Rifky Almadani
NIM : 2510817310018

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1: Implementasi Stack 1
A. Source Code 1
B. Pembahasan 2
SOAL 2: Si Palui dan Mesin Hitung Aneh 4
A. Source Code 5
B. Output Program 6
C. Pembahasan 6
SOAL 3: Implementasi Queue 9
A. Source Code 9
B. Pembahasan 10
SOAL 4: Si Palui dan Deretan Angka 13
A. Source Code 14
B. Output Program 15
C. Pembahasan 15
TAUTAN GIT 17

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 2 6
Gambar 2 Screenshot Hasil Jawaban Soal 4 15

DAFTAR TABEL

Tabel 1 Source Code stack.cpp..... 1
Tabel 2 Source Code prob1.cpp 5
Tabel 3 Source Code queue.cpp 9
Tabel 4 Source Code prob2.cpp 14

SOAL 1: Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

bool isFull(const Stack* s) {
    if (isEmpty(s)) return false;
    return s->top == &(s->data[MAX - 1]);
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::out_of_range("Stack Overflow"); //
    }
    if (isEmpty(s)) {
        s->top = &(s->data[0]);
    } else {
        s->top++;
    }
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw std::out_of_range("Stack Underflow"); //
    }
```

```
33      if (s->top == &(s->data[0])) {
34          s->top = nullptr;
35      } else {
36          s->top--;
37      }
38  }
39
40  int peek(const Stack* s) {
41      if (isEmpty(s)) {
42          throw std::out_of_range("Stack is Empty"); //
43      }
44      return *(s->top);
45  }
```

B. Pembahasan

Pada baris [1], `#include "stack.h"` merupakan preprocessor directive yang digunakan untuk menyertakan file header buatan sendiri yang kemungkinan berisi deklarasi struct `Stack`, array `data`, pointer `top`, serta konstanta `MAX`.

Pada baris [2], `#include <stdexcept>` digunakan untuk menyertakan library yang menyediakan kelas exception seperti `std::out_of_range` untuk menangani error.

Pada baris [4–6], `void init(Stack* s)` merupakan fungsi untuk inisialisasi stack, di mana pointer `top` diatur menjadi `nullptr` yang menandakan bahwa stack masih kosong.

Pada baris [8–10], `bool isEmpty(const Stack* s)` merupakan fungsi yang digunakan untuk mengecek apakah stack kosong, yaitu dengan memeriksa apakah `top` bernilai `nullptr`.

Pada baris [12–15], `bool isFull(const Stack* s)` merupakan fungsi untuk mengecek apakah stack penuh. Jika stack kosong maka langsung mengembalikan `false`. Jika tidak, maka dicek apakah pointer `top` sudah berada di elemen terakhir array `(data[MAX - 1])`.

Pada baris [17–27], `void push(Stack* s, int value)` merupakan fungsi untuk menambahkan elemen ke dalam stack. Jika stack penuh, maka akan melempar exception `std::out_of_range` dengan pesan "Stack Overflow". Jika stack kosong, maka `top`

diarahkan ke elemen pertama array. Jika tidak, pointer `top` digeser ke atas (increment). Setelah itu nilai `value` disimpan pada posisi yang ditunjuk oleh `top`.

Pada baris [29–38], `void pop(Stack* s)` merupakan fungsi untuk menghapus elemen teratas dari stack. Jika stack kosong, maka akan melempar exception "Stack Underflow". Jika elemen yang dihapus adalah elemen terakhir (posisi awal array), maka `top` dikembalikan menjadi `nullptr`. Jika tidak, pointer `top` digeser ke bawah (decrement).

Pada baris [40–45], `int peek(const Stack* s)` merupakan fungsi untuk melihat nilai elemen teratas dari stack tanpa menghapusnya. Jika stack kosong, maka akan melempar exception "Stack is Empty". Jika tidak kosong, maka nilai yang ditunjuk oleh `top` akan dikembalikan.

SOAL 2: Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s

Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

N

A1 A2 . . . An

Keterangan:

- Baris pertama n menyatakan jumlah atau Panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

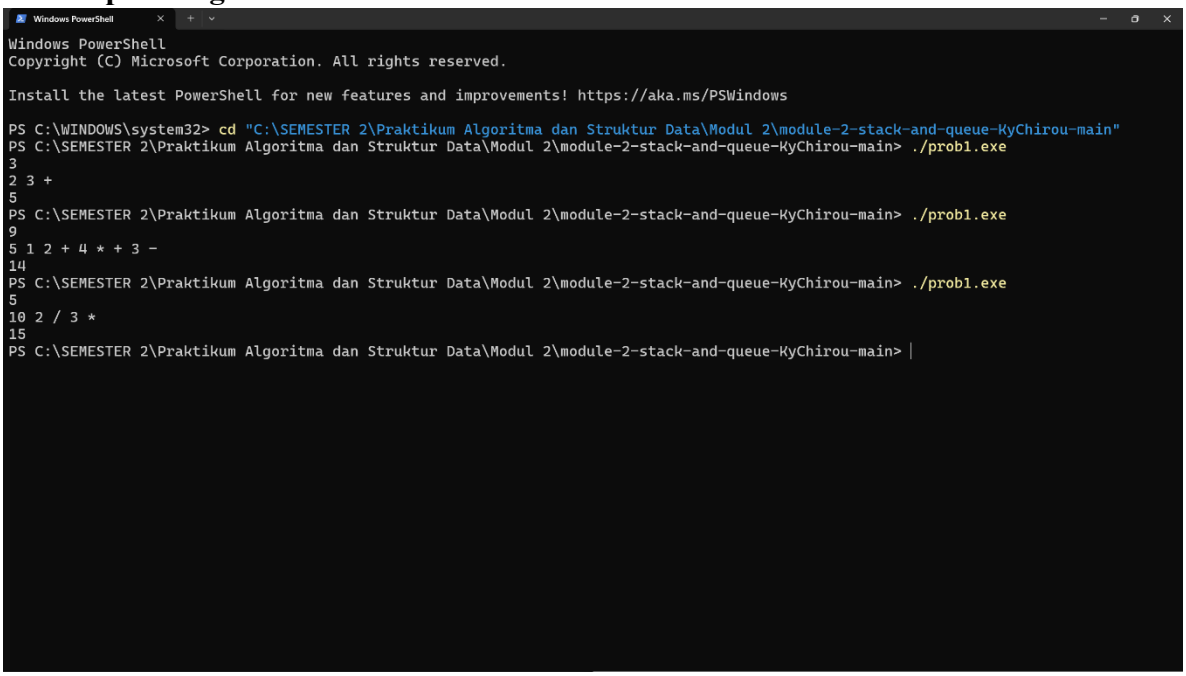
A. Source Code

Tabel 2 Source Code prob1.cpp

```
#include "stack.h"
#include <iostream>
#include <string>

int main() {
    int n;
    if (!(std::cin >> n)) return 0;

    Stack s;
    init(&s);

    for (int i = 0; i < n; i++) {
        std::string token;
        std::cin >> token;

        if (isdigit(token[0]) || (token.length() > 1 && token[0] == '-')) {
            push(&s, std::stoi(token));
        } else {
            int val2 = peek(&s); pop(&s);
            int val1 = peek(&s); pop(&s);

            if (token == "+") push(&s, val1 + val2);
            else if (token == "-") push(&s, val1 - val2);
            else if (token == "*") push(&s, val1 * val2);
```

```
25                  else if (token == "/") push(&s, val1 / val2);
26              }
27          }
28
29          std::cout << peek(&s) << std::endl;
30          return 0;
31      }
```

B. Output Program



Gambar 1 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [1], `#include "stack.h"` merupakan preprocessor directive yang digunakan untuk menyertakan file header yang berisi deklarasi struktur `Stack` serta fungsi-fungsi seperti `init`, `push`, `pop`, dan `peek`.

Pada baris [2], `#include <iostream>` digunakan untuk menyertakan library input-output yang menyediakan `std::cin` dan `std::cout`.

Pada baris [3], `#include <string>` digunakan untuk menyertakan library string agar dapat menggunakan tipe data `std::string`.

Pada baris [5], `int main()` merupakan fungsi utama yang menjadi titik awal eksekusi program.

Pada baris [6], `int n;` merupakan deklarasi variabel yang digunakan untuk menyimpan jumlah input (token) yang akan diproses.

Pada baris [7], `if (!(std::cin >> n)) return 0;` digunakan untuk membaca input jumlah token. Jika input gagal, maka program langsung berhenti dengan `return 0`.

Pada baris [9], `Stack s;` merupakan deklarasi variabel `s` bertipe `Stack`.

Pada baris [10], `init(&s);` digunakan untuk menginisialisasi stack agar dalam kondisi kosong.

Pada baris [12–27], `for (int i = 0; i < n; i++)` merupakan perulangan yang akan berjalan sebanyak `n` kali untuk membaca dan memproses setiap token.

Pada baris [13], `std::string token;` merupakan deklarasi variabel string untuk menyimpan input berupa angka atau operator.

Pada baris [14], `std::cin >> token;` digunakan untuk membaca token dari input.

Pada baris [16], `if (isdigit(token[0]) || (token.length() > 1 && token[0] == '-'))` merupakan kondisi untuk mengecek apakah token adalah bilangan (termasuk bilangan negatif).

Pada baris [17], `push(&s, std::stoi(token));` digunakan untuk mengubah string menjadi integer menggunakan `std::stoi`, lalu memasukkannya ke dalam stack.

Pada baris [18], `else` menandakan bahwa token adalah operator.

Pada baris [19], `int val2 = peek(&s); pop(&s);` digunakan untuk mengambil nilai teratas dari stack sebagai operand kedua, lalu menghapusnya dari stack.

Pada baris [20], `int val1 = peek(&s); pop(&s);` digunakan untuk mengambil nilai berikutnya sebagai operand pertama, lalu menghapusnya dari stack.

Pada baris [22], `if (token == "+") push(&s, val1 + val2);` digunakan untuk melakukan operasi penjumlahan dan menyimpan hasilnya kembali ke stack.

Pada baris [23], `else if (token == "-") push(&s, val1 - val2);` digunakan untuk operasi pengurangan.

Pada baris [24], `else if (token == "*") push(&s, val1 * val2);` digunakan untuk operasi perkalian.

Pada baris [25], `else if (token == "/") push(&s, val1 / val2);` digunakan untuk operasi pembagian.

Pada baris [29], `std::cout << peek(&s) << std::endl;` digunakan untuk menampilkan hasil akhir yang berada di puncak stack.

Pada baris [30], `return 0;` digunakan untuk mengakhiri program dengan status sukses.

SOAL 3: Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3 Source Code queue.cpp

```
#include "queue.h"
#include <iostream>
#include <stdexcept>

void init(Queue* q) {
    q->front = nullptr;
    q->rear = nullptr;
}

bool isEmpty(const Queue* q) {
    return q->front == nullptr;
}

bool isFull(const Queue* q) {
    if (isEmpty(q)) return false;
    return (q->front == &(q->data[0]) && q->rear == &(q->data[MAX - 1]));
}

void enqueue(Queue* q, int value) {
    if (isFull(q)) {
        throw std::out_of_range("Queue Overflow");
    }

    if (isEmpty(q)) {
        q->front = &(q->data[0]);
        q->rear = &(q->data[0]);
        *(q->rear) = value;
    }
    else if (q->rear == &(q->data[MAX - 1])) {
        int size = q->rear - q->front + 1;
        for (int i = 0; i < size; i++) {
```

```
32                 q->data[i] = q->front[i];
33             }
34             q->front = &(q->data[0]);
35             q->rear = &(q->data[size - 1]);
36 
37             q->rear++;
38             *(q->rear) = value;
39         }
40         else {
41             q->rear++;
42             *(q->rear) = value;
43         }
44 }
45 
46 void dequeue(Queue* q) {
47     if (isEmpty(q)) {
48         throw std::out_of_range("Queue Underflow");
49     }
50 
51     if (q->front == q->rear) {
52         q->front = nullptr;
53         q->rear = nullptr;
54     } else {
55         q->front++;
56     }
57 }
58 
59 int front(const Queue* q) {
60     if (isEmpty(q)) {
61         throw std::out_of_range("Queue is Empty");
62     }
63     return *(q->front);
64 }
65 
66 int back(const Queue* q) {
67     if (isEmpty(q)) {
68         throw std::out_of_range("Queue is Empty");
69     }
70     return *(q->rear);
71 }
```

B. Pembahasan

Pada baris [1], `#include "queue.h"` merupakan preprocessor directive yang digunakan untuk menyertakan file header buatan sendiri yang berisi deklarasi struct `Queue`, array `data`, pointer `front` dan `rear`, serta konstanta `MAX`.

Pada baris [2], `#include <iostream>` digunakan untuk menyertakan library input-output, meskipun pada kode ini tidak secara langsung digunakan.

Pada baris [3], `#include <stdexcept>` digunakan untuk menyertakan library exception yang memungkinkan penggunaan `std::out_of_range` untuk menangani error.

Pada baris [5–8], `void init(Queue* q)` merupakan fungsi untuk inisialisasi queue, di mana pointer `front` dan `rear` diatur menjadi `nullptr` yang menandakan queue masih kosong.

Pada baris [10–12], `bool isEmpty(const Queue* q)` merupakan fungsi untuk mengecek apakah queue kosong, yaitu ketika `front` bernilai `nullptr`.

Pada baris [14–17], `bool isFull(const Queue* q)` merupakan fungsi untuk mengecek apakah queue penuh. Jika queue kosong maka langsung mengembalikan `false`. Jika tidak, maka queue dianggap penuh ketika `front` berada di awal array (`data[0]`) dan `rear` berada di akhir array (`data[MAX - 1]`).

Pada baris [19–44], `void enqueue(Queue* q, int value)` merupakan fungsi untuk menambahkan elemen ke dalam queue.

Pada baris [20–22], dilakukan pengecekan apakah queue penuh. Jika penuh, maka akan melempar exception "Queue Overflow".

Pada baris [24–27], jika queue kosong, maka `front` dan `rear` diarahkan ke elemen pertama array, kemudian nilai dimasukkan ke posisi tersebut.

Pada baris [29–38], kondisi ketika `rear` sudah berada di akhir array. Untuk mengatasi hal ini, dilakukan pergeseran elemen ke depan (shifting) agar ada ruang kosong di belakang. Variabel `size` digunakan untuk menghitung jumlah elemen, kemudian elemen dipindahkan ke awal array. Setelah itu, pointer `front` dan `rear` diperbarui, lalu elemen baru ditambahkan.

Pada baris [40–42], jika masih ada ruang di belakang array, maka pointer `rear` cukup digeser ke kanan (increment), kemudian nilai baru dimasukkan.

Pada baris [46–57], `void dequeue(Queue* q)` merupakan fungsi untuk menghapus elemen dari depan queue.

Pada baris [47–49], jika queue kosong, maka akan melempar exception "Queue Underflow".

Pada baris [51–56], jika hanya terdapat satu elemen (front sama dengan rear), maka setelah dihapus queue menjadi kosong sehingga kedua pointer diatur kembali ke `nullptr`. Jika tidak, maka pointer `front` digeser ke kanan.

Pada baris [59–64], `int front(const Queue* q)` merupakan fungsi untuk mengambil nilai elemen paling depan tanpa menghapusnya. Jika queue kosong, maka akan melempar exception, jika tidak maka mengembalikan nilai yang ditunjuk oleh `front`.

Pada baris [66–71], `int back(const Queue* q)` merupakan fungsi untuk mengambil nilai elemen paling belakang tanpa menghapusnya. Jika queue kosong, maka akan melempar exception, jika tidak maka mengembalikan nilai yang ditunjuk oleh `rear`.

SOAL 4: Si Palui dan Deretan Angka

Time Limit: 1.0 s

Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelah, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$

- $-1000 \leq A_i \leq 1000$

Keterangan:

- n: jumah elemen
- k: ukuran jendela
- A_i: elemen array

Format Input

n k

A1 A2 . . . An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

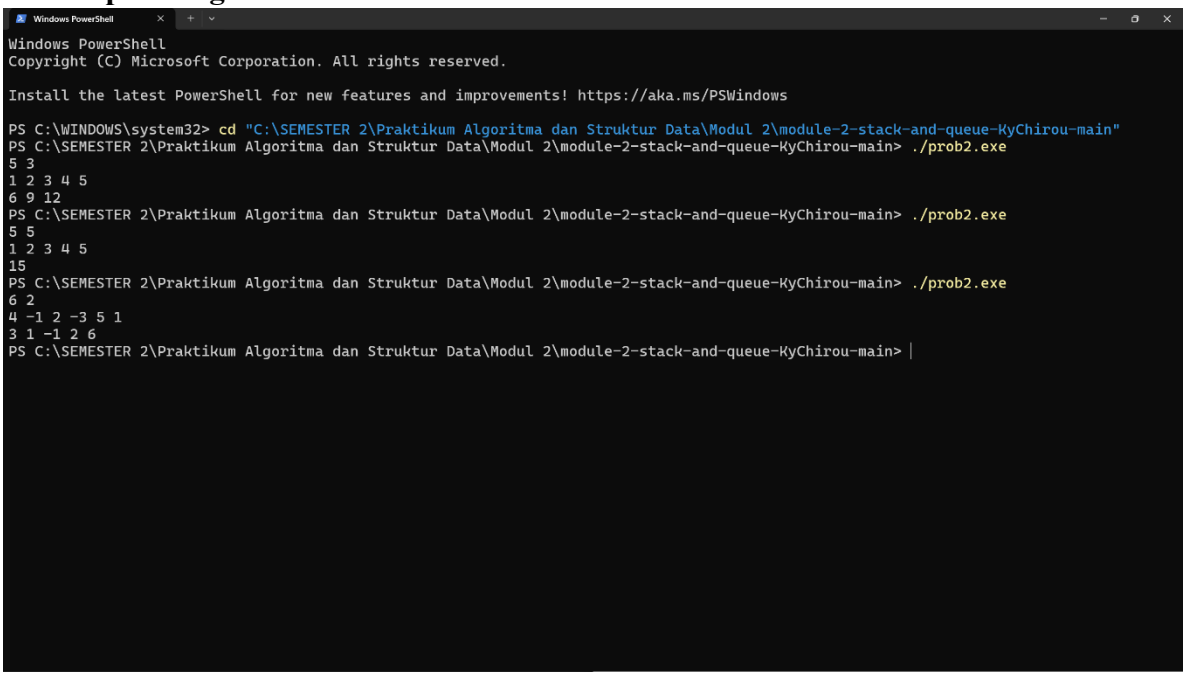
A. Source Code

Tabel 4 Source Code prob2.cpp

```
#include "queue.h"
#include <iostream>

int main() {
    int n, k;
    if (!(std::cin >> n >> k)) return 0;

    int arr[105];
    for (int i = 0; i < n; i++) {
        std::cin >> arr[i];
    }

```

```
    for (int i = 0; i <= n - k; i++) {
        Queue q;
        init(&q);
        int current_sum = 0;

        for (int j = i; j < i + k; j++) {
            enqueue(&q, arr[j]);
            current_sum += arr[j];
        }

        std::cout << current_sum;

        if (i < n - k) {
            std::cout << " ";
        }
    }
    std::cout << std::endl;
    return 0;
}
```

B. Output Program



Gambar 2 Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Pada baris [1], `#include "queue.h"` merupakan preprocessor directive yang digunakan untuk menyertakan file header yang berisi deklarasi struktur Queue serta fungsi-fungsi seperti `init` dan `enqueue`.

Pada baris [2], `#include <iostream>` digunakan untuk menyertakan library input-output yang menyediakan `std::cin` dan `std::cout`.

Pada baris [4], `int main()` merupakan fungsi utama yang menjadi titik awal eksekusi program.

Pada baris [5], `int n, k;` merupakan deklarasi variabel untuk menyimpan jumlah elemen array (`n`) dan ukuran jendela atau jumlah elemen yang akan dijumlahkan (`k`).

Pada baris [6], `if (!(std::cin >> n >> k)) return 0;` digunakan untuk membaca input `n` dan `k`. Jika input gagal, maka program langsung dihentikan.

Pada baris [8], `int arr[105];` merupakan deklarasi array untuk menyimpan hingga 105 elemen bilangan bulat.

Pada baris [9–11], perulangan `for` digunakan untuk mengisi array `arr` dengan `n` buah input dari pengguna.

Pada baris [13–28], perulangan `for (int i = 0; i <= n - k; i++)` digunakan untuk melakukan proses pergeseran jendela (sliding window) sepanjang array, dari indeks awal hingga indeks `n - k`.

Pada baris [14], `Queue q;` merupakan deklarasi variabel queue untuk menyimpan elemen sementara dalam setiap jendela.

Pada baris [15], `init(&q);` digunakan untuk menginisialisasi queue agar kosong sebelum digunakan.

Pada baris [16], `int current_sum = 0;` digunakan untuk menyimpan jumlah elemen dalam jendela saat ini.

Pada baris [18–21], perulangan `for (int j = i; j < i + k; j++)` digunakan untuk mengambil `k` elemen berturut-turut dari array mulai dari indeks `i`.

Pada baris [19], `enqueue(&q, arr[j]);` digunakan untuk memasukkan elemen ke dalam queue.

Pada baris [20], `current_sum += arr[j];` digunakan untuk menjumlahkan elemen-elemen dalam jendela.

Pada baris [23], `std::cout << current_sum;` digunakan untuk menampilkan hasil jumlah dari setiap jendela.

Pada baris [25–27], kondisi `if (i < n - k)` digunakan untuk menambahkan spasi antar output, kecuali pada hasil terakhir.

Pada baris [29], `std::cout << std::endl;` digunakan untuk berpindah ke baris baru setelah semua hasil ditampilkan.

Pada baris [31], `return 0;` digunakan untuk mengakhiri program dengan status sukses.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-KyChirou